Chapter01

소프트웨어 개발과 깃 허브



GitHub : cse-yhpark

유한대학교 컴퓨터소프트웨어공학과

박 영 환

목차

- 소프트웨어 개발
- CI/CD
- 깃허브
- 환경 구축

GitHub : cse-yhpark

Section01

소프트웨어 개발과 깃 허브



GitHub : cse-yhpark

유한대학교 컴퓨터소프트웨어공학과

박 영 환

소프트웨어 개발과 깃허브

- **소프트웨어 개발 분야의 의의 및 특성**
  - 현대사회 및 비즈니스 필수 핵심 분야
    - 높은 난이도와 수많은 시행착오 수반 작업 특성
    - 끊임없는 기술 발전 대응을 위한 지속적 학습 의무화

- **소프트웨어 엔지니어의 일과 가치 전달**
  - 소프트웨어를 통한 사용자 가치 전달 수행(중심 활동 : 프로그래밍)
  - 단순 코드 작성의 한계 극복을 위한 필수 활동 3가지 수행
    - 코드를 소프트웨어로 변환 및 제대로 실행되는지 검증
    - 소프트웨어를 사용자가 사용할 수 있는 상태로 조성

GitHub : tsse-vhpark



소프트웨어 개발의 특성 - 1

- **보편적 필요성과 문제 영역(CI/CD 도입 배경)**
  - 소프트웨어 종류 불문 보편적 필요 활동에 대한 CI/CD 적용
  - 소프트웨어 개발 자체가 지닌 본질적 틀징 이해 선행

- **복잡성과 변화 (Complexity & Change)**
  - 다수 요소의 상호 영향에 따른 복잡성 및 예상치 못한 동작발생
  - 사용자 니즈 다양화 및 시장 환경·요건의 일상적 변화 대응

GitHub : ese-yhpark

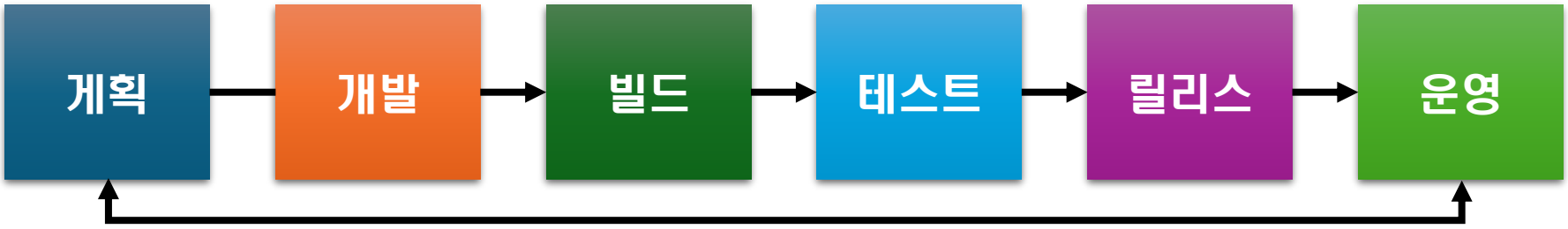
- **품질과 기대치(Quality & Expectation)**
  - 단순 실행 여부를 넘어 가볍고 빠른 실행 요구
  - 정보 누설 방지 등 사용자 프라이버시 보호 필수 준수(미부응 시외면 초래)

소프트웨어 개발의 특성 - 2

- **피할 수 없는 시행착오 지속적(Continuous) 진행**
  - 사전 모든 계획 원천적 불가능성
  - 반복적 시행착오를 통한 원하는 결과 도발 방식 채택
  - '반복' 특성에 기인한 지속적(Continuous) 개발 성격 형성

- **소프트웨어 개발 주기(SDLC)**
  - 빌드, 테스트, 릴리스, 운영 등의 복잡한 스탭 순환 구조 형성
  - 해당 주기를 반복하며 사용자에게 가치를 저달하는 활도 전개

GitHub : ese-yhpark



Section02

CI/CD



GitHub : cse-yhpark

유한대학교 컴퓨터소프트웨어공학과

박 영 환

CI/CD의 개념 : CI와 CD - 1

- **CI/CD 자동화 개요**
  - '빌드'부터 '릴리스'까지의 단계 자동화 구현
  - 자동화를 통합 각 스텝의 지속적 실행 및 시행착오 속도·횟수 증대

- **지속적 통합(CI : Continuous Integration)**
  - 정의
    - 변경한 코드를 코드베이스에 반영하고 검증하는 작업 (검증 과정이 핵심)
  - 수행 과정 및 필요성
    - 코드 변경 사항의 빈번한 코드베이스 통합 및 제대로 실행되지 반복 검증
    - 코드 통합 빈도 증가를 통한 충돌(Conflict) 감소 유도
    - 반복 검증에 따른 버그 초기 발견 및 신속한 수정, 안정적 실행 도모

GitHub : cse-yhpark

CI/CD의 개념 : CI와 CD - 2

- **지속적 전달(CD : Continuous Delivery)**
  - 소프트에 생성 이후 사용자 전달을 위한 릴리스(release) 수행
  - 릴리스의 특성
    - 사람의 실수 유발 용이, 필요한 작업의 다양성, 복잡한 순서, 단일성 이벤트가 아닌 반복 작업 특성
  - 언제나 안전하게 릴리스할 수 있는 상태 유지 및 반복적 개선 실행
  - 핵심 요소
    - 자동화
      - 반복 작업 강화, 사람의 관여 여지 축소로 언제든 안전한 사용자 전달 보장
    - CI와 CD의 관계 : CI가 CD에 포함되는 구조
      - 높은 품질의 소프트웨어를 신속히 제공하는 역할 포괄

GitHub : cse-yhpark

CI/CD와 지속 가능성

- **소프트웨어 개발의 지속 가능성 (Sustainability)**
  - **목적**
    - **사용자에게 가치 제공이라는 사명에 '시간 축' 개념을 추가하여 장기적 가치 제공 ➔ 소프트웨어 공학 전체의 목적**
  - **사용자 만족도 향상 혜택**
    - **지속적 테스트로 초기 스텝 버그 발견, 품질 향상 및 위험 최소화**
    - **정상 실행 소프트웨어 지속 전달을 통한 사용자 신뢰 획득 및 이탈 방지**
    - **릴리스 자동화를 통한 안정적 전달 및 피드백 기반 지속 개선**

GitHub : ese-yhpark

CI/CD와 지속 가능성

- **CI/CD 도입의 다방면 혜택**
  - **비즈니스 경쟁력 향상:**
    - **사용자/시장 급변에 유연 대처, 피드백 주기 반복을 통한 비즈니스 기회 창출, 적시 제공, 가설 검증 능력 확보**
  - **소프트웨어 엔지니어의 창의성 향상**
    - **조기 문제 발견 및 대처로 실패를 두려워하지 않는 도전 환경 조성**
    - **자동화를 통한 반복 작업 대폭 축소, 수작업 오류 및 복구 시간 단축**
    - **여유 시간 확보를 통한 학습, 실험, 신규 도전, 동기 부여 및 개발 민첩성 향상**

GitHub : ssss yhpark

CI/CD의 혜택과 버전 관리

- **버전 관리 시스템 (VCS : Version Control System) 도입 전제**
  - **필수 전제 조건**
    - VCS 도입 (**코드 보관 및 체인지로그 관리, 임의 시점 되돌리기 가능**)
  - **대표적 도구 : 깃**(Git)
  - **개발 규모별 관리 방식**
    - 1**인 개발**
      - **자신의** PC**에서만 버전 관리 수행 가능**
    - 2**팀 개발**
      - **코드를 원격 접속 가능한 곳에 보관하여 공동 버전 관리 수행**
        ➔ 깃허브 등장 배경

GitHub : cse-yhpark

Section03

깃허브



GitHub : cse-yhpark

유한대학교 컴퓨터소프트웨어공학과

박 영 환

깃허브 개요 및 부가 서비스

- **깃허브(GitHub) 플랫폼 서비스 개요**
  - **소프트웨어 개발 전체 지원 플랫폼**
  - **원격 서버 코드 보관 및 공유 호스팅 기능 제공**
  - **깃(Git) 지원으로 여러 명이 공동 버전 관리 수행**
  - **기업 및 오픈소스 채택 세계적 서비스**
  - **풀 리퀘스트(Pull Request)를 통한 코드 변경 제안 및 리뷰 기능 제공**

GitHub : cse-yhpark

- **깃허브의 대표적 다수 부가 서비스**
  - **깃허브 액션(GitHub Actions) : 범용 워크플로 엔진**
  - **깃허브 코드스페이스(GitHub Codespaces) : 클라우드 기반 개발 환경**
  - **깃허브 코파일럿(GitHub Copilot) : 지능형 코드 보완 도구**

깃허브 계정 및 요금제

- **깃허브 계정의 종류**
  - **개인 계정**(Personal Account)
    - **가입 시 기본 제공, 리포지터리, 패키지, 프로젝트 등 리소스**
  - **제공조직 계정**(Organization Account)
    - **여러 개인 계정 포함, 리소스 제공 및 사용자/권한 관리 수행**
  - **엔터프라이즈 계정**(Enterprise Account)
    - **여러 조직 계정 포함 (엔터프라이즈 요금제 가입 시 사용)**

GitHub : cse-yhpark

- **깃허브 요금제 특징**
  - **깃허브 프리**(GitHub Free)
    - **무료 요금제, 대다수 개인 계정 및 기능 제한된 조직 계정 사용**
  - **깃허브 프로**(GitHub Pro)
    - **개인 계정용 유료 요금제 (무료 제한 기능 사용 가능)**
  - **깃허브 팀**(GitHub Team)
    - **조직 계정용 유료 요금제 (기업 실무 다수 활용)**
  - **깃허브 엔터프라이즈**(GitHub Enterprise)
    - **최상위 유료 요금제 (SSO 통합 인증 등 고급 기능 제공)**

Scetion04

환경 구축



GitHub : cse-yhpark

유한대학교 소프트웨어공학과

박 영 환

리포지터리 종류 및 공식 문서

- **리포지터리(Repository) 종류 및 특징**
  - **퍼블릭 리포지터리**(Public Repository)
    - **인터넷 공개, 누구나 접근 가능, 개인 취미 및 오픈소스 프로젝트 적합, 무료 요금제에서도 기능 제한 거의 없음**
  - **프라이빗 리포지터리**(Private Repository)
    - **지정된 사람만 접근 가능 (조직 계정 시 그룹 속 인원 접근), 비즈니스 코드 비공개 용도 주 사용, 무료 요금제 시 기능 제한 다수**
  - **인터널 리포지터리**(Internal Repository)
    - **엔터프라이즈 계정 속 그룹끼리 공유 가능한 프라이빗의 일종**

GitHub : cse-yhpark

- **깃허브 공식 문서 활용 방법**
  - **공식 사이트 주소**
    - docs.github.com/ko (**의문점 발생 시 검색 활용, 주요 검색 키워드 영어 병기**)
  - **주의 사항**
    - **문서 참조 시 왼쪽 상단** [Version] **확인 필수**
    - **엔터프라이즈 요금제용 문서 별도 존재**

환경 구축 및 깃허브 CLI

- **실습 환경 전제 조건**
  - 깃(Git) 설치 필수
  - OS 종류 무관
    - 명령줄은 Bash 전제 설명 윈도우는 파워셸PowerShell 별도 표기

- **깃허브 CLI (Command-Line Interface) 개요**
  - 명령줄에서 깃허브 작업을 처리하는 도구
  - 주요 처리 가능 작업
    - 풀 리퀘스트 작성, 코멘트 추가, 병합(merge) 처리
    - 깃허브 액션 참조, 실행, 유효화, 무효화 제어
    - 리포지터리 작성, 설정, 포크(fork), 아카이브(archive) 관리

- **일상적 작업의 명령줄 실행을 통한 개발 작업의 쾌적함 및 자동화 향상**

GitHub : ese-yhpark

깃허브 CLI 설치 및 로그인

- **깃허브 CLI 설치 방법 (OS별)**
  - macOS **홈브루**(Homebrew) **이용**: ‘brew install gh’
  - **윈도우 패키지 관리자**(winget) **이용**: ‘winget install --id GitHub.cli’

- **설치 후 버번 확인 및 인증 절차**
  - **버전 확인 명령어** : ‘gh version’
  - **로그인 명령어** : ‘gh auth login’ (**실행 시 브라우저 연동**, ID **및 정보 입력**)
  - **최초 로그인 시** one-time code **확인 및 입력**

GitHub : ose-yhpark

- **사용법 확인 명령어**
  - **도움말 확인**: ‘gh help’

리포지터리 작성

- **깃허브 CLI 활용 명령어 실습**
  - **리포지터리 목록 확인**: `gh repo list`
  - **현재 브랜치에서 풀 리퀘스트 작성**
    - 'gh pr create --fill –web' (--web **플래그로 브라우저 실행 결과 확인 가능**)
  - **자주 사용하는 명령어는 앨리어스**(Alias) **등록 추천**

- **깃허브 CLI를 이용한 리포지터리 작성**
  - **기능 제한이 적은 퍼블릭 리포지터리 추천** (my-repo **명칭 변경 가능**)
  - **생성 명령어 예시**
    - 'gh repo create my-repo --public --clone --add-readme'

GitHub : cse-yhpark